

😊 Add icon 🖼️ Add cover

Untitled

What they are describing is a very common real-world cloud engineering maturity problem:

“We built fast to enable delivery, but now we need to redesign the identity and permissions model properly using least privilege, separation of duties, and environment isolation.”

This is actually an excellent teaching moment for:

- DevSecOps maturity
- Enterprise application governance
- CI/CD identity security
- Managed identities
- RBAC strategy
- Environment segmentation
- Zero Trust implementation

Here’s how you can explain the **best-practice architecture and remediation approach** clearly and professionally during the workshop.

▼ What the Current Situation Sounds Like

They currently have:

- A SaaS product
- Hosted on VMs
- Migrating to Azure PaaS
- CI/CD pipelines
- Enterprise Applications (Service Principals)
- Automation accounts/pipelines
- Permissions granted broadly
- Overprivileged deployment identities
- Weak environment separation

This usually means:

	≡ Problem	≡ Risk
1	One giant pipeline identity	Massive blast radius
2	Shared permissions across environments	Dev compromise affects prod
3	Excessive Contributor/Owner access	Privilege escalation
4	Static secrets in pipelines	Credential theft
5	No workload identity separation	Lateral movement
6	Fast delivery over governance	Long-term operational risk

▼ The Correct Security Best-Practice Explanation

1. Separate Environments Properly

You should explain:

“Development, staging, testing, and production must be treated as separate trust boundaries.”

Best practice:

	≡ Environment	≡ Isolation Level
1	Dev	Lowest trust
2	Test	Medium trust
3	Stage	Pre-production
4	Production	Highest trust

Recommended separation:

- Separate subscriptions where possible
- Separate resource groups minimum
- Separate identities
- Separate pipelines
- Separate Key Vaults
- Separate RBAC scopes

▼ 2. Separate Identities Per Pipeline

Right now they likely have:

- One Enterprise App
- One Service Principal
- One huge permission model

That is NOT best practice.

Instead:

	≡ Pipeline	≡ Identity
1	Dev deployment pipeline	Dev SPN/Managed Identity
2	Stage deployment pipeline	Stage identity
3	Prod deployment pipeline	Production identity

Why?

Because:

- Compromise stays isolated
- Permissions stay scoped
- Auditing improves
- Conditional Access becomes possible
- Least privilege becomes enforceable

▼ 3. Use Least Privilege RBAC

This is the biggest concept.

Explain:

“The pipeline identity should only have the exact permissions required for the deployment action.”

Example:

BAD:

- Owner at subscription level

BETTER:

- Contributor at resource-group level

BEST:

- Custom role with exact deployment permissions
-

▼ 4. Move Away from Secrets

They likely use:

- Client secrets
- Static credentials
- Pipeline secrets

Best practice:

- Managed Identities
- Federated workload identities
- OIDC federation
- Azure DevOps/GitHub workload identity federation

This eliminates:

- Secret rotation problems
 - Credential leakage
 - Long-lived secrets
-

▼ 5. Treat Enterprise Applications as Production Security Assets

This is critical.

Most organizations:

- Create Enterprise Apps quickly
- Never review permissions
- Never rotate credentials
- Never monitor usage

Explain:

“Enterprise Applications are effectively privileged machine identities and must be governed like administrator accounts.”

Best practice:

- Naming standards
 - Ownership assignment
 - Access reviews
 - Logging
 - Conditional Access
 - PIM where possible
 - Certificate-based auth
 - Secret expiration governance
-

▼ 6. Introduce DevSecOps Governance

Right now:

"Make it work first."

That's normal early-stage engineering.

Now maturity requires:

	≡ Old Model	≡ Mature Model
1	Fast deployment	Secure deployment
2	Shared access	Scoped access
3	Static secrets	Federated identity
4	Manual governance	Policy-driven governance
5	Reactive security	Shift-left security

7. Recommended Azure Architecture

You can explain something like this:

Plain Text ▾

1 Management Group

2 |

3 |— Production Subscription

4 | |— Prod Resource Groups

5 | |— Prod Key Vault

6 | |— Prod Managed Identities

7 | |— Prod CI/CD Pipeline Identity

8 |

9 |— Staging Subscription

10 | |— Stage Resources

11 | |— Stage Deployment Identity

12 |

13 |— Development Subscription

14 | |— Dev Resources

15 | |— Dev Deployment Identity

16

8. Security Controls They Should Add

Identity Controls

- Conditional Access
- MFA for privileged admins
- PIM
- Managed identities

Pipeline Controls

- Separate pipelines
- Environment approvals
- Manual approval gates
- Signed artifacts
- Secretless auth

Governance Controls

- Azure Policy

- Defender for Cloud
- Secure Score
- RBAC reviews
- Access reviews

Monitoring Controls

- Sentinel
- Defender XDR
- Audit logs
- Service principal sign-in logs

▼ 9. The Key Security Principle

This is the cleanest executive explanation:

“The goal is to reduce blast radius.”

Meaning:

- If Dev gets compromised → Production survives
- If one pipeline identity gets compromised → Entire tenant is not exposed
- If one secret leaks → All environments are not compromised

▼ 10. How This Aligns with Microsoft Best Practice

This directly aligns with:

- Zero Trust
- Microsoft CAF
- Well-Architected Framework
- Defender for Cloud
- Entra workload identity guidance
- DevSecOps maturity models
- Least privilege architecture

▼ Suggested Instructor Explanation (Short Version)

“What you’re describing is actually a very common cloud maturity journey.

Initially, teams build quickly to enable delivery and automation. The focus is speed, functionality, and getting the platform operational.

But over time, those automation identities and enterprise applications become highly privileged. If one pipeline identity is compromised, the attacker may gain access across development, staging, and production environments.

Best practice is to redesign the environment around Zero Trust and least privilege principles.

That means:

- Separate environments properly
- Separate deployment identities
- Scope permissions narrowly
- Replace static secrets with managed identities or federated identities
- Introduce RBAC governance
- Add approval gates and policy controls
- Monitor service principal activity continuously

The key objective is reducing blast radius.

A compromise in development should never automatically compromise production.”

Practical Azure Portal Demo: Fix Overprivileged CI/CD Enterprise App

Demo Goal

Show how to move from **one overprivileged enterprise application** to a **least-privilege, environment-separated deployment model**.

Use this storyline:

“We currently have one big CI/CD identity. We will split it into Dev, Stage, and Prod identities, scope permissions correctly, and remove unnecessary access.”

Microsoft RBAC assigns access to users, groups, service principals, or managed identities at a defined scope, such as subscription, resource group, or resource. ([Microsoft Learn](#))

▼ Demo 1 — Show the Problem: One Enterprise App Has Too Much Access

Portal Steps

1. Go to **Azure Portal**
2. Search for **Subscriptions**
3. Select the target subscription
4. Go to **Access control (IAM)**
5. Select **Role assignments**
6. Search for the existing CI/CD enterprise app or service principal
7. Show if it has:
 - **Owner**
 - **Contributor**
 - Access at **subscription scope**
 - Access across Dev, Stage, and Prod

Instructor Talk Track

“This is the risk. One automation identity can deploy everywhere. If this credential is leaked or abused, the attacker could impact all environments. Our goal is to reduce blast radius.”

▼ Demo 2 — Create Separate Resource Groups

Portal Steps

1. Go to **Resource groups**
2. Create:
 - `rg-saas-dev`
 - `rg-saas-stage`
 - `rg-saas-prod`
3. Use same region for demo simplicity
4. Add tags:
 - `Environment = Dev`
 - `Environment = Stage`

- Environment = Prod
- Owner = PlatformTeam
- CostCenter = SecurityWorkshop

Instructor Talk Track

“Minimum isolation is separate resource groups. Better isolation is separate subscriptions. Production should always be treated as a separate trust boundary.”

▼ Demo 3 — Create Separate App Registrations / Enterprise Apps

Portal Steps

1. Go to Microsoft Entra ID
2. Select App registrations
3. Select New registration
4. Create:
 - spn-saas-dev-deploy
 - spn-saas-stage-deploy
 - spn-saas-prod-deploy
5. For each app:
 - Supported account type: Single tenant
 - Redirect URI: leave blank unless required

Instructor Talk Track

“Each environment now has its own deployment identity. Dev identity deploys only to Dev. Stage identity deploys only to Stage. Production identity deploys only to Production.”

▼ Demo 4 — Assign Least-Privilege RBAC

Portal Steps

1. Go to Resource groups
2. Open rg-saas-dev
3. Select Access control (IAM)
4. Select Add role assignment
5. Choose role:
 - For demo: Contributor
 - For best practice: custom deployment role
6. Select Members
7. Choose User, group, or service principal
8. Search for spn-saas-dev-deploy
9. Assign the role

Repeat:

	≡ Resource Group	≡ Identity	≡ Role
1	rg-saas-dev	spn-saas-dev-deploy	Contributor

2	<code>rg-saas-stage</code>	<code>spn-saas-stage-deploy</code>	Contributor
3	<code>rg-saas-prod</code>	<code>spn-saas-prod-deploy</code>	Contributor or custom role

Instructor Talk Track

“Notice that we are assigning access at the resource group level, not the subscription level. This limits what the pipeline identity can touch.”

Azure RBAC role assignments are the mechanism used to grant access at a selected scope. ([Microsoft Learn](#))

▼ Demo 5 — Remove Excessive Access from the Old Identity

Portal Steps

1. Go to **Subscriptions**
2. Open the target subscription
3. Go to **Access control (IAM)**
4. Select **Role assignments**
5. Search for the old enterprise app
6. Select the role assignment
7. Click **Remove**

Instructor Talk Track

“This is the remediation moment. We are removing broad subscription-level access and replacing it with scoped deployment identities.”

▼ Demo 6 — Show Workload Identity Federation Instead of Secrets

Portal Steps

1. Go to **Microsoft Entra ID**
2. Select **App registrations**
3. Open `spn-saas-dev-deploy`
4. Go to **Certificates & secrets**
5. Point out:
 - Client secrets create rotation risk
 - Secrets can leak in pipelines
6. Go to **Federated credentials**
7. Select **Add credential**
8. Choose scenario:
 - **GitHub Actions deploying Azure resources**, or
 - **Other issuer** for Azure DevOps/service connection scenarios
9. Add repository/project details
10. Save

Instructor Talk Track

“The modern best practice is to avoid long-lived secrets. Workload identity federation allows the pipeline to exchange a trusted token for access without storing a client secret.”

Microsoft describes workload identity federation as a trust relationship between Microsoft Entra and an external identity provider, allowing supported workloads to access protected resources without managing secrets. ([Microsoft Learn](#))

▼ Demo 7 — Add Production Approval Gate Concept

In Azure DevOps

1. Go to Pipelines
2. Select Environments
3. Create:
 - dev
 - stage
 - prod
4. Open prod
5. Add Approvals and checks
6. Add required approver:
 - Security Lead
 - Platform Owner
 - Change Manager

Instructor Talk Track

“Even if the production deployment identity exists, production deployment should require approval. Identity isolation controls access; approval gates control release risk.”

Azure DevOps supports Azure Resource Manager service connections with workload identity federation for deployments. ([Microsoft Learn](#))

▼ Demo 8 — Show Auditability

Portal Steps

1. Go to Microsoft Entra ID
2. Select Enterprise applications
3. Search for spn-saas-dev-deploy
4. Open the app
5. Review:
 - Sign-in logs
 - Audit logs
 - Permissions
 - Owners

Instructor Talk Track

“Service principals and enterprise applications must be treated like privileged identities. They need owners, logs, reviews, and lifecycle governance.”

▼ Final Architecture to Show

- Plain Text ▾
- 1 Dev Pipeline

2 ↓

```
3  spn-saas-dev-deploy
4      ↓
5  rg-saas-dev  only
6
7  Stage Pipeline
8      ↓
9  spn-saas-stage-deploy
10     ↓
11 rg-saas-stage  only
12
13 Prod Pipeline
14     ↓
15 spn-saas-prod-deploy
16     ↓
17 rg-saas-prod  only
18     ↓
19 Approval  required
20
```



▼ Closing Best-Practice Message

“The best practice is not just to make the pipeline work. The best practice is to make it work securely, with separate identities, scoped permissions, no long-lived secrets, approval gates, and full auditability.”

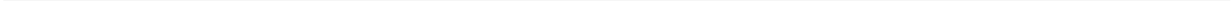
This second scenario is actually a very strong **enterprise identity architecture** discussion.

What they are describing is:

- Centralized identity management
- Enterprise Applications
- App Registrations
- GitLab SSO integration
- SAML/OIDC federation
- Security group-based authorization
- Customer tenant isolation
- Internal vs external trust boundaries
- Multi-tenant access architecture
- Azure Entra identity federation
- Internal productivity integrations (Teams/Jira/Zabbix)
- Dev-created app registrations
- Tenant hardening and governance

This is an excellent opportunity to demonstrate:

- Zero Trust identity architecture
- Enterprise Application governance
- App Registration security
- SSO best practices
- External identity federation
- Tenant segmentation
- Conditional Access
- Least privilege identity design



▼ What They Are Actually Describing

They have:

	≡ Component	≡ Description
1	GitLab	Connected to Entra ID
2	Enterprise Apps	Used for SSO integrations
3	App Registrations	Used for custom integrations
4	Security Groups	Control access authorization
5	Customer-facing tenant services	Segregated from internal operations
6	Internal integrations	Teams/Jira/Zabbix/Dev tooling
7	Multi-tenant complexity	Separate identity domains
8	Dev-created apps	Potential governance risk

▼ The Core Security Concern

This line is the important one:

“The dev team creates app registrations.”

This is extremely common.

And extremely dangerous without governance.

Because app registrations can:

- Read mail
- Access Teams
- Access SharePoint
- Read users
- Access Graph API
- Request OAuth scopes
- Become privileged workload identities

So this becomes:

▼ “How do we govern machine identities and SSO integrations securely?”

▼ Practical Portal Demo — Enterprise Applications & App Registrations Security

▼ Demo 1 — Show Enterprise Applications vs App Registrations

Instructor Goal

Explain the difference clearly.

Portal Steps

Go to:

- 1. Azure Portal
- 2. Microsoft Entra ID

Show:

App Registrations

Navigate:

- App registrations

Explain:

"This is the identity definition and authentication configuration."

Show:

Enterprise Applications

Navigate:

- Enterprise applications

Explain:

"This is the operational service identity instance used inside the tenant."

▼ Instructor Whiteboard Explanation

Plain Text ▾

1 App Registration

2 ↓

3 Defines identity/authentication

4

5 Enterprise Application

6 ↓

7 Operational service identity inside tenant

8

▼ Critical Teaching Point

	☰ App Registration	☰ Enterprise Application
1	Developer object	Operational security object
2	Defines OAuth/SAML/OIDC	Defines access in tenant
3	API permissions	User assignments
4	Authentication config	Conditional Access
5	Multi-tenant capable	Tenant-specific instance

▼ Demo 2 — Show GitLab SSO Integration

Portal Steps

- Go to:
 - Enterprise Applications
- Search:
 - GitLab
- Open GitLab Enterprise App

Show These Sections

Single Sign-On

Explain:

- SAML
- OIDC
- Federation
- Token exchange

Users and Groups

Explain:

- Security groups control authorization
- Group-based access is best practice

Instructor Talk Track

“Instead of assigning users individually, we assign security groups. This improves governance, scalability, and auditing.”

▼ Demo 3 — Show Security Group-Based Authorization

Portal Steps

- Go to:
 - Entra ID
- Open:
 - Groups
- Create:
 - gitlab-dev-users
 - gitlab-admins
 - gitlab-readonly

Go Back

- Enterprise Applications

- 2. GitLab
- 3. Users and Groups
- 4. Add Assignment

Assign:

- `gitlab-dev-users`

▼ Best Practice Explanation

BAD:

- Individual user assignments

GOOD:

- Group assignments

BEST:

- Dynamic groups + PIM + Conditional Access

▼ Demo 4 — Show Dangerous API Permissions

This is one of the BEST demos.

Portal Steps

- 1. App Registrations
- 2. Open custom app
- 3. API Permissions

Show Examples

Dangerous:

- Directory.ReadWrite.All
- User.Read.All
- Mail.Read
- Files.Read.All

▼ Instructor Talk Track

“This is where many organizations accidentally create excessive privilege exposure.”

Explain:

- OAuth scopes
- Graph permissions
- Consent risk
- Tenant-wide access

▼ Demo 5 — Show Admin Consent Risk

Portal Steps

Inside API Permissions:

- Show "Grant admin consent"

▼ Explain

"One admin consent action can potentially grant tenant-wide access to sensitive data."

This becomes critical for:

- Dev-created applications
- Third-party SaaS
- Automation tools
- AI integrations

▼ Demo 6 — Restrict Who Can Create App Registrations

THIS is one of the highest-value governance demos.

Portal Steps

1. Entra ID
2. User Settings
3. App registrations

Show Setting

"Users can register applications"

If enabled:

- Any developer can create OAuth apps
- Any developer can request permissions
- Shadow integrations appear

▼ Best Practice

Instead:

- Disable broad app registration
- Create approval process
- Use dedicated app registration admins
- Review permissions

▼ Instructor Talk Track

"This is one of the biggest identity governance gaps in many organizations."

▼ Demo 7 — Show Conditional Access for Enterprise Apps

Portal Steps

1. Entra ID
2. Protection
3. Conditional Access
4. New Policy

Example Policy

Target:

- GitLab Enterprise App

Conditions:

- Require MFA
- Require compliant device
- Block risky sign-ins
- Restrict countries

▼ Explain

“Enterprise applications should not bypass your identity security model.”

▼ Demo 8 — Show Sign-In Logs for Enterprise Apps

Portal Steps

1. Enterprise Applications
2. GitLab
3. Sign-in logs

▼ Show

- Failed logins
- Geographic anomalies
- MFA enforcement
- Risk detections

▼ Explain

“Enterprise Applications are security boundaries and should be monitored continuously.”

▼ Demo 9 — Show App Owners Governance

Portal Steps

- 1. App Registration
- 2. Owners

▼ Explain

Every app should have:

- Named owner
- Business owner
- Technical owner
- Expiry review

▼ Best Practice Architecture to Explain

Plain Text ▾

1 Users
2 ↓
3 Security Groups
4 ↓
5 Enterprise Applications
6 ↓
7 SSO Federation (SAML/OIDC)
8 ↓
9 GitLab / Jira / Zabbix / Teams
10

▼ Governance Model to Explain

	≡ Area	≡ Best Practice
1	App registrations	Restricted creation
2	Enterprise apps	Centrally governed
3	API permissions	Reviewed regularly
4	SSO	Federated auth only
5	Access	Group-based
6	MFA	Mandatory
7	Conditional Access	Enforced
8	Monitoring	Sign-in logs + Sentinel
9	Secrets	Certificates or federation
10	Ownership	Mandatory
11	Customer separation	Tenant isolation

▼ High-Value Security Message

This is the key executive explanation:

“Applications are identities now.”

Meaning:

- APIs are identities
- Pipelines are identities
- AI agents are identities
- Integrations are identities

And therefore:

- They require governance
- Monitoring
- Least privilege
- Lifecycle management
- Conditional Access
- Auditing

▼ Final Security Maturity Message

You can close with this:

“What you’re describing is a mature federated identity architecture, but the challenge now becomes governance and control.

As organizations integrate GitLab, Jira, Teams, monitoring platforms, automation systems, and customer-facing services into Entra ID, applications themselves become privileged identities.

The biggest security risk is not only human users anymore — it is overprivileged applications, excessive OAuth permissions, unmanaged app registrations, and weak identity governance.

Best practice is to:

- Separate internal and customer trust boundaries
- Use group-based authorization
- Restrict who can create app registrations
- Review API permissions regularly
- Apply Conditional Access to enterprise apps
- Monitor sign-in logs continuously
- Use least privilege for workload identities
- Govern all applications like privileged accounts

Modern cloud security is identity security.”

▼ Microsoft Best Practice — Enterprise Applications, App Registrations & SSO Governance

This architecture aligns strongly with Microsoft Zero Trust identity guidance and Microsoft Entra governance principles.

The core Microsoft principle is:

“Assume breach. Verify explicitly. Use least privilege.”

In modern cloud environments:

- Applications are identities
- Pipelines are identities
- APIs are identities
- AI agents are identities
- Integrations are identities

That means:

- They must be governed like privileged accounts
- They require Conditional Access
- They require monitoring
- They require lifecycle management
- They require least privilege RBAC
- They require identity governance

Microsoft guidance for workload identities and enterprise applications emphasizes least privilege, governance, and secure federation models. [Microsoft Entra Workload Identities](#)

▼ Microsoft Best-Practice Architecture

Recommended Identity Architecture

Plain Text ▾

1

Users

2

↓

3

Security Groups

4

↓

5

Conditional Access

6

↓

7

Enterprise Applications

8

↓

9

SSO Federation (SAML/OIDC/OAuth)

10

↓

11

GitLab / Jira / Zabbix / Teams / APIs

12

▼ Microsoft Recommended Security Controls

	≡ Area	≡ Microsoft Best Practice
1	App registrations	Restrict creation
2	Enterprise apps	Centrally governed
3	OAuth permissions	Admin-reviewed
4	Access assignment	Group-based
5	Authentication	SAML/OIDC federation
6	MFA	Mandatory
7	Conditional Access	Required
8	Secrets	Replace with certificates/federation
9	Monitoring	Sign-in logs + Sentinel
10	Governance	Access reviews
11	Workload identities	Least privilege
12	Customer isolation	Tenant separation
13	Privileged access	PIM-enabled

▼ Practical Microsoft Entra Demo (Step-by-Step)

▼ Demo 1 — Explain App Registrations vs Enterprise Applications

Instructor Goal

Clarify the architecture.

Portal Steps

Navigate To:

- 1. Azure Portal
- 2. [Microsoft Entra Admin Center](#)

Show:

App Registrations

Navigate:

- Identity
- Applications
- App registrations

Explain

“App Registrations define authentication, OAuth scopes, certificates, APIs, and federation configuration.”

Show:

Enterprise Applications

Navigate:

- Identity
- Applications
- Enterprise applications

Explain

“Enterprise Applications are the operational service identities inside the tenant.”

▼ Instructor Whiteboard

- 1 App Registration
- 2 ↓
- 3 Identity Definition
- 4
- 5 Enterprise Application
- 6 ↓
- 7 Operational Security Object
- 8

▼ Demo 2 — Create Secure GitLab SSO Group Model

Portal Steps

Go To:

1. Identity
2. Groups
3. New Group

Create:

	≡ Group Name	≡ Purpose
1	gitlab-dev-users	Standard developers
2	gitlab-admins	Elevated GitLab admins
3	gitlab-readonly	Read-only users

Explain

“Microsoft recommends role-based and group-based authorization instead of assigning permissions directly to users.”

Microsoft RBAC and group-based access reduce administrative overhead and improve governance consistency. [Microsoft RBAC Overview](#)

▼ Demo 3 — Configure Enterprise App Access

Portal Steps

1. Enterprise Applications
2. Select GitLab
3. Users and Groups
4. Add user/group

Assign:

- gitlab-dev-users

▼ Explain

BAD:

- Direct user assignment

GOOD:

- Security group assignment

BEST:

- Dynamic groups + PIM + Conditional Access

▼ Demo 4 — Show Dangerous OAuth Permissions

This is one of the most important security demos.

Portal Steps

1. App registrations
2. Select custom app
3. API permissions

Show Dangerous Permissions

	☰ Dangerous Permission	☰ Risk
1	Directory.ReadWrite.All	Full directory modification
2	User.Read.All	Read all users
3	Mail.Read	Read mailboxes
4	Files.Read.All	Access SharePoint/OneDrive
5	Group.ReadWrite.All	Modify groups

▼ Instructor Talk Track

“This is where application identities become dangerous. Many organizations accidentally grant tenant-wide access to applications.”

▼ Demo 5 — Show Admin Consent Risk

Portal Steps

Inside:

- API Permissions

Show:

- “Grant admin consent”

▼ Explain

“One admin consent action can potentially authorize tenant-wide data access.”

This is especially dangerous for:

- Third-party SaaS
- Dev-created apps
- Automation tools
- AI integrations

Microsoft recommends careful governance around admin consent workflows. [Microsoft Admin Consent Governance](#)

▼ Demo 6 — Restrict App Registration Creation

Portal Steps

1. Identity
2. User settings
3. App registrations

Show:

“Users can register applications”

▼ Explain

If enabled:

- Developers create shadow apps
- OAuth sprawl occurs
- Governance weakens
- Excessive permissions appear

▼ Microsoft Best Practice

Instead:

- Disable broad creation
- Create approval workflow
- Use dedicated app admins
- Require security review

▼ Instructor Talk Track

“One of the largest enterprise identity risks today is unmanaged application sprawl.”

▼ Demo 7 — Configure Conditional Access for Enterprise Apps

Portal Steps

- 1. Protection
- 2. Conditional Access
- 3. New policy

Target:

- GitLab Enterprise App

Configure:

Conditions

- Require MFA
- Require compliant device
- Restrict countries
- Block risky sign-ins

▼ Explain

“Enterprise applications should inherit the same Zero Trust protections as users.”

Microsoft Zero Trust guidance strongly recommends Conditional Access as a primary identity enforcement layer. [Microsoft Zero Trust Identity Guidance](#)

▼ Demo 8 — Monitor Enterprise Application Sign-Ins

Portal Steps

- 1. Enterprise Applications
- 2. GitLab
- 3. Sign-in logs

Show:

- Failed logins
- Geographic anomalies
- Impossible travel
- MFA status
- Risk detections

▼ Explain

“Applications must be monitored continuously like privileged accounts.”

▼ Demo 9 — Secure Workload Identities

Portal Steps

1. App registrations
2. Certificates & secrets

Explain

BAD:

- Long-lived client secrets

GOOD:

- Certificates

BEST:

- Workload identity federation

▼ Show:

Federated Credentials

Explain:

- GitHub Actions federation
- GitLab federation
- OIDC trust
- Secretless authentication

Microsoft recommends workload identity federation to eliminate long-lived secrets. [Workload Identity Federation](#)

▼ Demo 10 — Show App Ownership Governance

Portal Steps

1. App registration
2. Owners

▼ Explain

Every application should have:

- Technical owner
- Business owner
- Expiry review
- Governance process
- Quarterly access review

▼ Microsoft Zero Trust Identity Principles

Verify Explicitly

- MFA
- Conditional Access

- Risk scoring

Use Least Privilege

- Group-based access
- Scoped permissions
- Minimal Graph scopes

Assume Breach

- Monitor apps continuously
- Restrict tenant-wide permissions
- Audit OAuth consent

Microsoft’s Zero Trust guidance applies equally to workload identities and applications. [Microsoft Zero Trust Architecture Center](#)

▼ Recommended Enterprise Governance Model

	≡ Governance Area	≡ Recommended Control
1	App registration creation	Restricted
2	OAuth permissions	Security reviewed
3	Admin consent	Approval workflow
4	Secrets	Eliminate where possible
5	Enterprise apps	Centrally governed
6	Access	Group-based
7	External SaaS	Federated SSO only
8	Monitoring	Sentinel integration
9	Customer environments	Tenant separation
10	Privileged apps	PIM + access reviews
11	Dev-created apps	Governance onboarding

▼ High-Value Executive Security Message

“Modern enterprises no longer only manage users — they manage application identities at scale.”

Explain:

- APIs authenticate
- Pipelines authenticate
- AI agents authenticate
- SaaS integrations authenticate

Therefore:

- Identity governance must include applications
- Zero Trust must include workload identities
- Monitoring must include enterprise apps
- Least privilege must apply to machine identities

▼ Final Instructor Closing Statement

“What you’re describing is already a fairly mature federated identity architecture.

The next maturity phase is governance.

As organizations integrate GitLab, Jira, Teams, monitoring platforms, automation systems, AI services, and customer-facing applications into Microsoft Entra, applications themselves become privileged identities.

The biggest risk is no longer only compromised users.

It is:

- Overprivileged applications
- Excessive OAuth permissions
- Unmanaged app registrations
- Weak admin consent governance
- Long-lived secrets
- Shadow integrations

Microsoft best practice is to treat all applications like privileged security principals.

That means:

- Least privilege
- Conditional Access
- MFA enforcement
- Group-based authorization
- Workload identity governance

- Continuous monitoring
- Ownership assignment
- Access reviews
- Secret elimination where possible

Modern cloud security is identity security.”